Aggregate

- Summarize data
- Consolidates the data
- extract specific data

Count() - we count unique identifier
Sum() - we sum numeric data (revenu
AVG() - we average numeric " "
MIN() - looks for min vole in a colum
MAX() - looks for max value ina colum

Group BY STATEMENT

gender, race, category

```
SELECT categorical_column,
    Aggregate_function (column) AS
        descriptive_column_name
FROM table_name numerical column
Group BY categorical_column
```

↳ we group by a column that not aggregate

HAVING STATEMENT

- it is used to filter data that is aggregated
- we apply the HAVING to data that is alread aggregated

Scanned with CamScanner

It is used to filter grouped results after using a GROUP BY

GROUP BY: categorical
WHERE

Sales

Cust_id	cust_name	product	amount
101	Rofhi	Laptop	5000
101	Rofhi	Mouse	300
102	Abel	Keyboard	700
102	Abel	Monitor	2500
103	Claro	Mouse	300

Write an SQL Query to find a customers who spent more than 3000

```
SELECT cust_name,
       SUM(amount) AS
    total_spent
From Sales
```

GROUP BY custor_name

Cust_name	Total_spent
Rathi	5000
Abel	3200
Ciara	300

For Ciara to be filtered we need to add the HAVING statement



```
SELECT customer_name
    Sum(amount) AS
      total_spen
FROM sales
GROUP BY custor name
HAVING Sum(amount) > 3000
```

Only group by: categorical column in your select statement

```
Select: categorical
        Colum
aggre fun(colum) AS
    descriptive_name
FROM Table
GROUP BY categorical Colm
```

Exercise

SQL Exercise : Aggregate Functions

Q1. COUNT()

```
SELECT COUNT * AS total_
       employees
       FROM Employees
```

= 10

Q2 SUM()

```
SELECT Sum (Salary)
       AS total_IT_Salary
FROM employees
WHERE department = 'IT'
```

total_IT_Salary
220 000

4. SELECT MAX (Salary) AS highest_salary,
MIN (salary) AS lowest_salary
FROM Employees

Highest salary	lowest salary
62 000	48 000

5. GROUP BY

SELECT department
Sum (salary) as total_salary
From employees
GROUP BY department

department	total Salary
IT	
FINANCE	
MARKETING	
HR	

6. GROUP BY

```
SELECT City, COUNT (Distinct id
    As employee_count
FROM employees
GROUP BY City
```

City	employee count
New York	1
Chicago	3
San Fran	2
Houston	1
Los Ang	2

7 Having Co

```
SELECT department,
    Sum(salary) AS
    total_salary
FROM employees
GROUP BY department
HAVING Sum Salary > 100 000;
```

department	total_salary
IT	220 000
Finan	119 000
Marketing	105 000

Scanned with CamScanner

- why not where clause

LIMIT STATEMENT

- It is a way of restricting the number of rows you want in your final output

Scanned with CamScanner

Case STATEMENTS

- Allows us to create a new column based on logical conditions applied on other columns
- Allows us to classify our numerical data in to categorical groupings

Syntax:

```
SELECT column1,
CASE
    WHEN condition THEN
    result1
    WHEN condition THEN
    result2
    ELSE default_result
    END AS column_name
FROM table_name
```

employees

emp_id	name	Salary
101	John	3000
102	Peter	7000
103	Samson	12000
104	David	4000
105	Goliyadi	300

Scanned with CamScanner

Business logic

Salary below 5000 - low

Salary between 5000 and 10000 - medium

Salary above 10 000 - high

```
SELECT name, salary

CASE
        WHEN salary < 5000
    THEN 'Low'
    WHEN Salary BETWEEN
    5000 AND 10000 THEN
    'medium'
    WHEN Salary > 10 000
     THEN 'HIGH'
  END AS salary_bracket
From employee
```

- The case statement is placed within the select statement
- Case statement creates a new column

name	salary	salary_bracket
John	3000	law
Peter	7000	med
Samson	12000	high
David	4000	law
Goliyadi	300	law

Task

Table products

product_id	product_name	price
1	Laptop	1200
2	Phone	800
3	Keyboard	45
4	Monito	300
5	Mouse	85

Classify

Expensive IF price > 100

Mid-range 100 and 1000

Budget < 100

Scanned with CamScanner

SELECT product_name, price

CASE WHEN price > 1000 THEN 'expensive'
WHEN BETWEEN 100 AND 1000 THEN 'mid-range'
WHEN price < 100 THEN 'Budget'

END AS price category

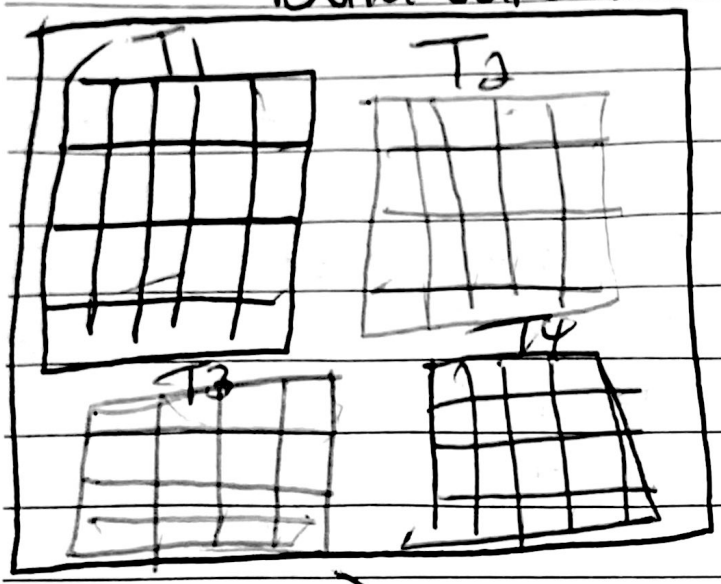
FROM ~~employees~~ products

product_name	price	price_category
Laptop	1200	exp
Phone	800	mid
Keyboard	45	Bud
Monitor	300	mid
Mouse	25	Bud

Scanned with CamScanner

SQL Joins

Datawarehause – Relational database



- The different table have a form of relation (multiple)

Datamanagement System (Databrics)

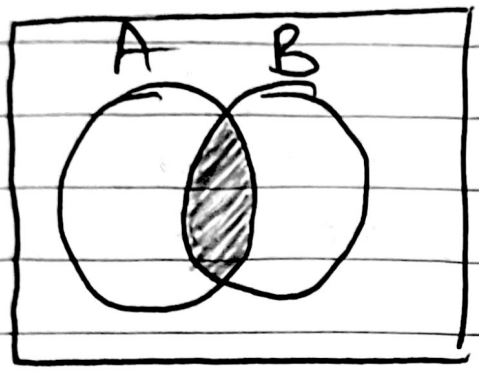
SQL Coding

↳ We can join multiple into one table

SQL Joins

↳ meant to join multiple tables based on common columns

Inner Join



- We join tables on columns but the join affect the output rows
- When we join tables we can select any column between the tables we are joining
- Returns rows where there is a match on both the tables

Syntax:

```
SELECT column1,
       colum2,
       column3,
       colum4
From  table1 AS A
   INNER JOIN table2
   AS B ON A.common_
   column = B.common_
   colum
```

Customers

cust_id	name
1	Rathi
2 X	Mbali

orders

order_id	cus_id	amou
101	1	500
102 X	4	20
103 X	3	10

```
SELECT A.cust_id,
       name,
       amount
From customers AS A
INNER Joint orders AS B
   ON  A.cust_id = B.custid
```

cust_id	name	amount
1	Rathi	500

- What gets returned is what matches on bothe the tables
- Ensures that the outcome is what is on both tables

Primary key	Foreign key
- we must look at the same	

cust — primary key in the table (unique identifier)

cust_id	name
1	Rd
2	Mbal

- different for each customer

Dimension table (static descriptive information) info doesn't change frequently

ord_id	cust_id	amount
101	1	500
102	4	20
103	5	10
104	1	300

Cust_id is a foreign key in orders (not unique) can be repeated

Fact table (numerical)

order table is primary key

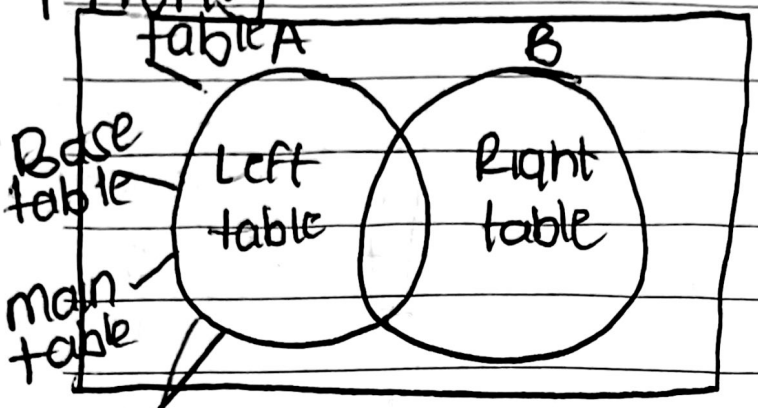
Outcome

cus_id	name	amount
1	Rdhi	500
	Rdhi	300

appears twice

LEFT JOIN

Priority



In our final outcome, we need all the rows in the left table

Syntax:

```
SELECT column1
       column2
       column3
FROM table1 AS A
LEFT JOIN table2 AS B
ON A.common_column =
   B.common_column
```

- A LEFT JOIN returns all records from the left table, and only the matching records from the right table
- When there is no match in the right table we still keep the left table now, but the right table columns will show NULL